

1. What is Backtracking?

Backtracking is an algorithmic technique used to solve problems by:

1. Trying all possible choices,
2. Moving forward recursively,
3. Undoing the choice if it does not lead to a solution,
4. Trying the next option.

It is basically:

Try → Explore → Undo → Try next

2. Simple Definition

Backtracking is:

A recursion technique where we revert changes after exploring each possibility.

3. Core Idea

At every step:

- Choose an option.
 - Go deeper using recursion.
 - If invalid or done, **backtrack** (undo).
 - Try next option.
-

4. General Flow

```
Start
↓
Choose option
↓
Recursive call
↓
Undo choice (Backtrack)
↓
Try next option
```

5. Generic Template

This template works for most problems.

C++ Template

```
void solve(...) {  
    if (base_condition) {  
        store_answer;  
        return;  
    }  
  
    for (all choices) {  
        choose;  
        solve(...);  
        unchoose; // backtrack  
    }  
}
```

6. Important Components

Component	Meaning
Choice	What options we can pick
Constraint	Valid/invalid move
Goal	Base condition
Undo step	Backtracking step

7. When to Use Backtracking?

Use backtracking when problem asks:

- All permutations
- All combinations
- All subsets
- All paths
- All valid arrangements
- Sudoku solving
- N-Queens
- Maze path
- Word search

Keyword: **“Find all possible solutions”**

LeetCode 46 — Permutations

Requirement followed:

- `res` → stores final result

- ds → temporary permutation
- mp → used to track used elements

☑ C++ Solution

```
class Solution {
public:
    void solve(vector<int>& nums, vector<vector<int>>& res,
               vector<int>& ds, vector<int>& mp) {

        if (ds.size() == nums.size()) {
            res.push_back(ds);
            return;
        }

        for (int i = 0; i < nums.size(); i++) {
            if (!mp[i]) {
                ds.push_back(nums[i]);
                mp[i] = 1;

                solve(nums, res, ds, mp);

                mp[i] = 0;
                ds.pop_back();
            }
        }
    }

    vector<vector<int>> permute(vector<int>& nums) {
        vector<vector<int>> res;
        vector<int> ds;
        vector<int> mp(nums.size(), 0);

        solve(nums, res, ds, mp);
        return res;
    }
};
```

☑ Java Solution

```
class Solution {

    void solve(int[] nums, List<List<Integer>> res,
               List<Integer> ds, boolean[] mp) {

        if (ds.size() == nums.length) {
            res.add(new ArrayList<>(ds));
            return;
        }

        for (int i = 0; i < nums.length; i++) {
            if (!mp[i]) {
                ds.add(nums[i]);
                mp[i] = true;

                solve(nums, res, ds, mp);
            }
        }
    }
}
```

```
        mp[i] = false;
        ds.remove(ds.size() - 1);
    }
}

public List<List<Integer>> permute(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    boolean[] mp = new boolean[nums.length];

    solve(nums, res, new ArrayList<>(), mp);
    return res;
}
}
```

☑ Python Solution

```
class Solution:
    def permute(self, nums):
        res = []
        ds = []
        mp = [False] * len(nums)

        def solve():
            if len(ds) == len(nums):
                res.append(ds.copy())
                return

            for i in range(len(nums)):
                if not mp[i]:
                    ds.append(nums[i])
                    mp[i] = True

                    solve()

                    mp[i] = False
                    ds.pop()

        solve()
        return res
```

Tc: $n! * n$

Sc: $O(n)$

LeetCode 46 — Permutations (Backtracking using Swap Approach)

In **swap-based backtracking**, we:

1. Fix one index.
2. Swap each element with the current index.
3. Recurse for the next index.
4. Swap back (backtracking).

Here:

- res stores answers.
- No mp needed because swapping handles usage.
- ds is the current array itself.

☑ C++ (Swap Backtracking)

```
class Solution {
public:
    void backtrack(int idx, vector<int>& ds,
                  vector<vector<int>>& res) {

        if (idx == ds.size()) {
            res.push_back(ds);
            return;
        }

        for (int i = idx; i < ds.size(); i++) {
            swap(ds[idx], ds[i]);

            backtrack(idx + 1, ds, res);

            swap(ds[idx], ds[i]); // backtrack
        }

        vector<vector<int>> permute(vector<int>& nums) {
            vector<vector<int>> res;
            vector<int> ds = nums;

            backtrack(0, ds, res);
            return res;
        }
    };
};
```

☑ Java (Swap Backtracking)

```
class Solution {

    void backtrack(int idx, int[] ds, List<List<Integer>> res) {
        if (idx == ds.length) {
            List<Integer> temp = new ArrayList<>();
            for (int num : ds) temp.add(num);
            res.add(temp);
            return;
        }
    }
}
```

```
        for (int i = idx; i < ds.length; i++) {
            int t = ds[idx];
            ds[idx] = ds[i];
            ds[i] = t;

            backtrack(idx + 1, ds, res);

            t = ds[idx];
            ds[idx] = ds[i];
            ds[i] = t;
        }
    }

    public List<List<Integer>> permute(int[] nums) {
        List<List<Integer>> res = new ArrayList<>();
        backtrack(0, nums, res);
        return res;
    }
}
```

☒ Python (Swap Backtracking)

```
class Solution:
    def permute(self, nums):
        res = []

        def backtrack(idx):
            if idx == len(nums):
                res.append(nums[:])
                return

            for i in range(idx, len(nums)):
                nums[idx], nums[i] = nums[i], nums[idx]

                backtrack(idx + 1)

                nums[idx], nums[i] = nums[i], nums[idx]

        backtrack(0)
        return res
```

☒ Swap Approach Advantage

- No extra space for mp
- Faster in practice
- Clean permutation generation
- Tc: $n! * n$
- Sc: $O(1)$

LeetCode 47 — Permutations II (Recursion + Backtracking)
(Handling duplicates)

We use the **same variables**:

- `res` → stores final permutations
- `ds` → temporary permutation
- `mp` → tracks used elements

Idea

1. Sort the array first.
2. Skip duplicates using condition:
3. `if(i > 0 && nums[i] == nums[i-1] && !mp[i-1]) continue;`
4. Use recursion + backtracking.

C++ Solution

```
class Solution {
public:
    void solve(vector<int>& nums, vector<vector<int>>& res,
               vector<int>& ds, vector<int>& mp) {

        if (ds.size() == nums.size()) {
            res.push_back(ds);
            return;
        }

        for (int i = 0; i < nums.size(); i++) {

            if (mp[i]) continue;

            if (i > 0 && nums[i] == nums[i - 1] && !mp[i - 1])
                continue;

            ds.push_back(nums[i]);
            mp[i] = 1;

            solve(nums, res, ds, mp);

            mp[i] = 0;
            ds.pop_back();
        }
    }

    vector<vector<int>> permuteUnique(vector<int>& nums) {
        sort(nums.begin(), nums.end());

        vector<vector<int>> res;
        vector<int> ds;
        vector<int> mp(nums.size(), 0);

        solve(nums, res, ds, mp);
        return res;
    }
};
```

☑ Java Solution

```
class Solution {

    void solve(int[] nums, List<List<Integer>> res,
               List<Integer> ds, boolean[] mp) {

        if (ds.size() == nums.length) {
            res.add(new ArrayList<>(ds));
            return;
        }

        for (int i = 0; i < nums.length; i++) {

            if (mp[i]) continue;

            if (i > 0 && nums[i] == nums[i - 1] && !mp[i - 1])
                continue;

            ds.add(nums[i]);
            mp[i] = true;

            solve(nums, res, ds, mp);

            mp[i] = false;
            ds.remove(ds.size() - 1);
        }
    }

    public List<List<Integer>> permuteUnique(int[] nums) {
        Arrays.sort(nums);

        List<List<Integer>> res = new ArrayList<>();
        boolean[] mp = new boolean[nums.length];

        solve(nums, res, new ArrayList<>(), mp);
        return res;
    }
}
```

☑ Python Solution

```
class Solution:
    def permuteUnique(self, nums):
        nums.sort()

        res = []
        ds = []
        mp = [False] * len(nums)

        def solve():
            if len(ds) == len(nums):
                res.append(ds.copy())
                return

            for i in range(len(nums)):
                if mp[i]:
```



```
        continue

    if i > 0 and nums[i] == nums[i-1] and not mp[i-1]:
        continue

    ds.append(nums[i])
    mp[i] = True

    solve()

    mp[i] = False
    ds.pop()

solve()
return res
```